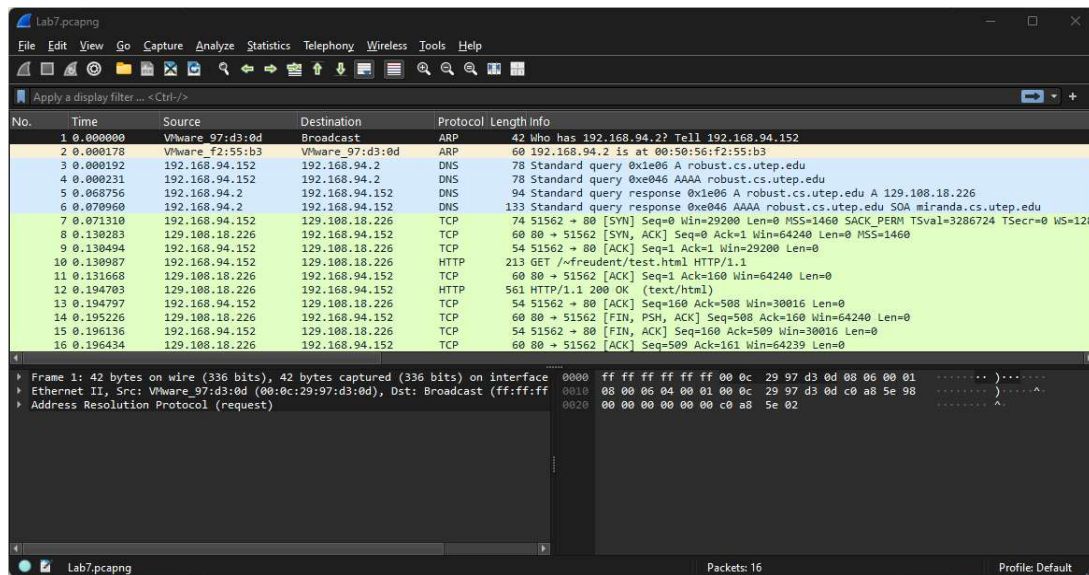


Lab 7

Stephanie Salgado

Opening the file on Wireshark



There are only 16 frames which makes important details easy to spot.

ARP traffic

Notice frames 1 and 2 are the only “ARP” protocol frames.

Request

```
Address Resolution Protocol (request)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: request (1)
  Sender MAC address: Vmware_97:d3:0d (00:0c:29:97:d3:0d)
  Sender IP address: 192.168.94.152
  Target MAC address: 00:00:00_00:00:00 (00:00:00:00:00:00)
  Target IP address: 192.168.94.2
```

The request was made in frame 1, this can be confirmed by double clicking the frame.

Response

```
Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: VMware_f2:55:b3 (00:50:56:f2:55:b3)
  Sender IP address: 192.168.94.2
  Target MAC address: VMware_97:d3:0d (00:0c:29:97:d3:0d)
  Target IP address: 192.168.94.152
```

The response was made in frame 2. Once again, it can be confirmed by inspecting the frame.

IP address being requested

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	Vmware 97:d3:0d	Broadcast	ARP	42	who has 192.168.94.2? Tell 192.168.94.152
2	0.000178	Vmware f2:55:h3	Vmware 97:d3:0d	ARP	60	192.168.94.2 is at 00:50:56:f2:55:h3

The request is asking "Who has 192.168.94.2?", and the response confirms that "192.168.94.2" is at MAC address "00:50:56:f2:55:b3".

Protocol layers

```

Frame 2: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on interface eno16777736, id 0
Ethernet II, Src: VMware_f2:55:b3 (00:50:56:f2:55:b3), Dst: VMware_97:d3:0d (00:0c:29:97:d3:0d)
  Destination: VMware_97:d3:0d (00:0c:29:97:d3:0d)
  Source: VMware_f2:55:b3 (00:50:56:f2:55:b3)
  Type: ARP (0x0806)
  [Stream index: 1]
  Padding: 00000000000000000000000000000000
Address Resolution Protocol (reply)
  Hardware type: Ethernet (1)
  Protocol type: IPv4 (0x0800)
  Hardware size: 6
  Protocol size: 4
  Opcode: reply (2)
  Sender MAC address: VMware_f2:55:b3 (00:50:56:f2:55:b3)
  Sender IP address: 192.168.94.2
  Target MAC address: VMware_97:d3:0d (00:0c:29:97:d3:0d)
  Target IP address: 192.168.94.152

```

The "Ethernet" (Layer 2) is used to deliver ARP frames across the local network. The "ARP" is between Layers 2 and 3 it's the protocol used to map IP addresses to MAC addresses. It can be considered as being Layer 2.5.

Conjecture for ARP

The ARP was generated because “192.168.94.152” needed to communicate with “192.168.94.2” but didn't know its MAC address. ARP resolved this by broadcasting the request.

DNS traffic

Frames 3-6 are DNS protocol.

DNS requests

```
Domain Name System (query)
Transaction ID: 0x1e06
Flags: 0x0100 Standard query
 0... .. = Response: Message is a query
.000 0... .. = Opcode: Standard query (0)
... ..0... .. = Truncated: Message is not truncated
... ..1... .. = Recursion desired: Do query recursively
... ..0... .. = Z: reserved (0)
... ..0... .. = Non-authenticated data: Unacceptable
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
  robust.cs.utep.edu: type A, class IN
    Name: robust.cs.utep.edu

Domain Name System (query)
Transaction ID: 0xe046
Flags: 0x0100 Standard query
 0... .. = Response: Message is a query
.000 0... .. = Opcode: Standard query (0)
... ..0... .. = Truncated: Message is not truncated
... ..1... .. = Recursion desired: Do query recursively
... ..0... .. = Z: reserved (0)
... ..0... .. = Non-authenticated data: Unacceptable
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
  robust.cs.utep.edu: type AAAA, class IN
    Name: robust.cs.utep.edu
```

Frames 3 and 4 are the requests for “address” records, IPv4 (A) and IPv6 (AAAA) respectively. They are queries for “robust.cs.utep.edu”.

DNS responses

```
Answers
  robust.cs.utep.edu: type A, class IN, addr 129.108.18.226
[Request In: 3]
```

```
Authoritative nameservers
  cs.utep.edu: type SOA, class IN, mname miranda.cs.utep.edu
    Name: cs.utep.edu
    Type: SOA (6) (Start Of a zone of Authority)
    Class: IN (0x0001)
    Time to live: 5 (5 seconds)
    Data length: 43
    Primary name server: miranda.cs.utep.edu
    Responsible authority's mailbox: hostmaster.cs.utep.edu
    Serial Number: 2010098619
    Refresh Interval: 10800 (3 hours)
    Retry Interval: 3600 (1 hour)
    Expire limit: 604800 (7 days)
    Minimum TTL: 86400 (1 day)
[Request In: 4]
```

The responses are found in frames 5 and 6. frame 6 provides a full response including an “SOA” record.

Hostname and IP

3	0.000192	192.168.94.152	192.168.94.2	DNS	78 Standard query 0x1e06 A robust.cs.utep.edu
4	0.000231	192.168.94.152	192.168.94.2	DNS	78 Standard query 0xe046 AAAA robust.cs.utep.edu
5	0.008756	192.168.94.2	192.168.94.152	DNS	94 Standard query response 0x1e06 A robust.cs.utep.edu A 129.108.18.226
6	0.070960	192.168.94.2	192.168.94.152	DNS	133 Standard query response 0xe046 AAAA robust.cs.utep.edu SOA miranda.cs.utep.edu

The DNS query in frame 3 is asking for the IP of “robust.cs.utep.edu” (hostname). The response in frame 5 provides the IP address as “129.108.18.226”, while frame 6 provides an additional SOA record.

Protocol layers

```
Ethernet II, Src: VMware_97:d3:0d (00:0c:29:97:d3:0d), Dst: VMware_f2:55:b3 (00:50:56:f2:55:b3)
Internet Protocol Version 4, Src: 192.168.94.152, Dst: 192.168.94.2
User Datagram Protocol, Src Port: 37298, Dst Port: 53
Domain Name System (query)
```

```
Ethernet II, Src: VMware_f2:55:b3 (00:50:56:f2:55:b3), Dst: VMware_97:d3:0d (00:0c:29:97:d3:0d)
Internet Protocol Version 4, Src: 192.168.94.2, Dst: 192.168.94.152
User Datagram Protocol, Src Port: 53, Dst Port: 37298
Domain Name System (response)
```

The protocol layers are the same for all these frames, with the only differences being that at Layer 7, there may be a “query” or a “response”. Layer 2, “Ethernet”, handles the frame transmission. Layer 3, “IP”, is used for addressing and routing the DNS packets. Layer 4, “UDP”, carries the DNS request and response, as DNS typically uses UDP for efficiency. Finally, the “DNS”, (Layer 7) is the application-layer protocol for domain name resolution.

Reason for the ARP

1	0.000000	VMware_97:d3:0d	Broadcast	ARP	42 Who has 192.168.94.2? Tell 192.168.94.152
2	0.000178	VMware_f2:55:b3	VMware_97:d3:0d	ARP	60 192.168.94.2 is at 00:50:56:f2:55:b3
3	0.000192	192.168.94.152	192.168.94.2	DNS	78 Standard query 0x1e06 A robust.cs.utep.edu

The DNS request was made by “192.168.94.2”. If the system needed to contact an IP like “129.108.18.226”, it might need to resolve the local gateway's MAC address via ARP first. This could explain why ARP traffic was generated.

HTTP traffic

URL being requested

+	10	0.130987	192.168.94.152	129.108.18.226	HTTP	213	GET /~freudent/test.html HTTP/1.1
---	----	----------	----------------	----------------	------	-----	-----------------------------------

The URL being requested is “/~freudent/test.html”. This can be seen in the information section for frame 10, the HTTP GET request.

Protocol layers

```
Ethernet II, Src: VMware_97:d3:0d (00:0c:29:97:d3:0d), Dst: VMware_f2:55:b3 (00:50:56:f2:55:b3)
Internet Protocol Version 4, Src: 192.168.94.152, Dst: 129.108.18.226
Transmission Control Protocol, Src Port: 51562, Dst Port: 80, Seq: 1, Ack: 1, Len: 159
Hypertext Transfer Protocol
  GET /~freudent/test.html HTTP/1.1\r\n
```

Here, the protocol layers are: Layer 2, “Ethernet” (transfers data over the local network), Layer 3, “IP” (ensures routing between the source and destination IPs), Layer 4, “TCP” (provides reliable transmission of HTTP traffic), and Layer 7, “HTTP” (the application-layer protocol used for web communication).

Frames establishing and closing a transport

7	0.071310	192.168.94.152	129.108.18.226	TCP	74	51562 → 80 [SYN] Seq=0 Win=29200 Len=0 MSS=1460 SACK_PERM
8	0.130283	129.108.18.226	192.168.94.152	TCP	60	80 → 51562 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
9	0.130494	192.168.94.152	129.108.18.226	TCP	54	51562 → 80 [ACK] Seq=1 Ack=1 Win=29200 Len=0
10	0.130987	192.168.94.152	129.108.18.226	HTTP	213	GET /~freudent/test.html HTTP/1.1
11	0.131668	129.108.18.226	192.168.94.152	TCP	60	80 → 51562 [ACK] Seq=1 Ack=160 Win=64240 Len=0
12	0.194703	129.108.18.226	192.168.94.152	HTTP	561	HTTP/1.1 200 OK (text/html)
13	0.194797	192.168.94.152	129.108.18.226	TCP	54	51562 → 80 [ACK] Seq=160 Ack=508 Win=30016 Len=0
14	0.195226	129.108.18.226	192.168.94.152	TCP	60	80 → 51562 [FIN, PSH, ACK] Seq=508 Ack=160 Win=64240 Len=0
15	0.196136	192.168.94.152	129.108.18.226	TCP	54	51562 → 80 [FIN, ACK] Seq=160 Ack=509 Win=30016 Len=0

Notice the TCP frames 7–9 and 14, 15. The TCP handshake starts in Frame 7 with “SYN”, used to initiate and establish a connection. In frame 8, there’s the “SYN-ACK”, which is a “SYN” message from the local device and “ACK” of the earlier packet. Frame 9 has another “acknowledgment”. The TCP teardown can be observed in frames 14 and 15, with “FIN-ACK”, in which “FIN” is used for terminating a connection.

Server

```
8 0.130283 129.108.18.226 192.168.94.152 TCP 60 80 → 51562 [SYN, ACK] Seq=885024273
```

IP Address: 129.108.18.226.

Port: 80 (standard HTTP port).

Initial Sequence Number: 885024273 or 0 (relative sequence numbers ON).

Client

```
7 0.071310 192.168.94.152 129.108.18.226 TCP 74 51562 → 80 [SYN] Seq=1156816002
```

IP Address: 192.168.94.152.

Port: 51562 (as seen in the TCP handshake in Frame 7).

Initial Sequence Number: 1156816002 or 0 (relative sequence numbers ON).

HTTP request

```
10 0.130987 192.168.94.152 129.108.18.226 HTTP 213 GET /~freudent/test.html HTTP/1.1
```

The HTTP request in frame 10 is a GET request from the client (192.168.94.152) asking for the resource “/~freudent/test.html” from the server at “robust.cs.utep.edu” (129.108.18.226).

HTTP ACK

```
11 0.131668 129.108.18.226 192.168.94.152 TCP 60 80 → 51562 [ACK] Seq=885024274 Ack=1156816162 Win=64240 Len=0
```

The “acknowledgment” (ACK) for the HTTP request is in frame 11. The “ACK” is coming from the server to let the client know the GET request was received. Notice it comes immediately after frame 10 in which the request was made in the first place.

HTTP headers

```
User-Agent: Wget/1.17.1 (linux-gnu)\r\nAccept: */*\r\nAccept-Encoding: identity\r\nHost: robust.cs.utep.edu\r\nConnection: Close\r\n\r\nDate: Sat, 23 Apr 2016 20:11:07 GMT\r\nServer: Apache/2.4.6 (CentOS) OpenSSL/1.0.1e-fips\r\nLast-Modified: Sat, 23 Apr 2016 19:55:25 GMT\r\nETag: "63-5312c5313e8bf"\r\nAccept-Ranges: bytes\r\n\r\n
```

The HTTP headers can be found in both frames 10 and 12.

HTTP response

```
12 0.194703 129.108.18.226 192.168.94.152 HTTP 561 HTTP/1.1 200 OK (text/html)
```

```
<title>test</title>\n<body>\n<H1> Eric's http test page</h2>\n<p> This is a <b>test</b>.\n</p>\n</body>\n
```

The HTTP 200 OK response is in frame 12, where the server returns the requested content.

Summary

For this lab, I used a provided Wireshark capture file to view ARP, DNS and HTTP traffic and identify the protocol layers involved. It was a capture file with only 16 frames. In the ARP response, I saw that IP "192.168.94.2" belonged to the MAC address "00:50:56:f2:55:b3". I was able to find why the ARP was generated after looking at the DNS traffic. The DNS requests and responses yielded the IP of "robust.cs.utep.edu". After that, I was able to find the URL requested (/~freudent/test.html) and view the TCP handshake and teardown. I was able to find server and client details, and view further HTTP traffic information like the request, "ACK", headers and response. Ultimately, this lab was good practice for Wireshark, especially since the file was so small. It made it easier to view and make assumptions based on the observable traffic.